Smart Contract Assessment

Defindex

HALBORN

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ABOUT THIS REPORT

ASSESSMENT TYPE
Assurance Assessment

DATE OF ENGAGEMENT
Aug 17, 2026 - Aug 20, 2026

SOURCE CODE
stellar-strategies

ASSESSED COMMIT ID
cdce87d

SEVERITY BREAKDOWN

20	15	4	1
All Findings	Solved	Risk Accepted	Acknowledged



INTRODUCTION

`Defindex` engaged Halborn to conduct a security assessment on their vault strategy contracts on Soroban beginning on August 17th, 2026 and ending on August 19th, 2026. The security assessment was scoped to the repository listed with commit hash, and further details in the **Scope** section of this report.

This protocol comprises two independent DeFindex vault strategy contracts on Stellar/Soroban that each implement the six-method DeFindexStrategyTrait to put vault-deposited capital to work in an upstream DeFi protocol.

KineticStrategy is a single-asset, supply-only autocompounding strategy that supplies a vault's deposited asset to the Kinetic (K2) lending protocol, holds the resulting aTokens, and lets a keeper role periodically claim reward emissions, swap them back to the principal asset via Soroswap, and re-supply them to grow NAV per share without minting new shares.

StableBondWrapperStrategy instead converts USDC deposits into a value-accruing RWA token (such as CETES, USTRY, or TESOURO Etherfuse stablebonds) via the Aquarius AMM, holds the RWA position, and converts it back to USDC on withdrawal, realizing RWA price appreciation as USDC-denominated NAV growth.

Both contracts share the same virtual-offset share-accounting model and are gated by a single keeper role that controls harvesting, depositor allowlisting, and risk parameters, with deposits restricted to allowlisted vault addresses or wasm code hashes while withdrawals remain open to any existing shareholder.

ASSESSMENT SUMMARY

The team at Halborn assigned one full-time security engineer to verify the security of the smart contracts. The security engineer is a blockchain and smart contract security expert with advanced penetration testing and smart contract hacking skills, and deep knowledge of multiple blockchain protocols.

The purpose of the assessment is to:

- Identify potential security issues within the different smart contracts and services.
- Ensure that smart contract and services functionality operates as intended.

In summary, Halborn identified some improvements to reduce the likelihood and impact of risks, which were mostly addressed by the `Defindex team`. The main recommendations were:

- `Build the router_pair_for call's arguments explicitly from only the first hop's two tokens instead of spreading the full multi-hop path array as flat positional arguments.`
- `Mint shares against the strategy's realized post-swap NAV increase rather than the pre-swap USDC amount deposited, so execution slippage is borne by the depositor alone.`
- `Vest harvested yield linearly into NAV, or add a minimum holding period or withdrawal fee, so capital timed around a harvest cannot capture a disproportionate share of the reward.`
- `Fold the strategy's idle underlying-asset balance into NAV, or add a keeper-only sweep method, so accidental direct transfers are not permanently stranded.`
- `Replace the same-pool Aqua quote used in the dual-source NAV check with a structurally independent pricing source that a single-transaction manipulation`

`cannot move.`

- `Restrict deposit authorization to the explicit per-address allowlist rather than trusting any contract instance that merely shares an allowlisted wasm code hash.`

TEST APPROACH AND METHODOLOGY

Halborn performed a combination of manual and automated security testing to balance efficiency, timeliness, practicality, and accuracy in regard to the scope of this assessment. While manual testing is recommended to uncover flaws in logic, process, and implementation; automated testing techniques help enhance coverage of the contracts' code and can quickly identify items that do not follow security best practices. The following phases and associated tools were used throughout the term of the assessment:

- Research into architecture and purpose.
- Modules manual code review and walk-through.
- Manual assessment of use and safety for the critical Soroban/Stellar variables and functions in scope to identify any arithmetic-related vulnerability classes.
- Local testing of Soroban contracts with the soroban-sdk test utilities.
- Static analysis of security for scoped contract, and imported functions.
- Dynamic analysis of critical functions.

Risk Methodology

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security

vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A)	1
	Specific (AO:S)	0.2
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

Critical	High	Medium	Low	Informational
9 - 10	7 - 8.9	4.5 - 6.9	2 - 4.4	0 - 1.9

Scope

REPOSITORY

(a) Repository: stellar-strategies

(b) Assessed Commit ID: cdce87d087ada57b4300f8fc830838f305e5cc5c

(c) Items in scope:

- contracts 12 files
  - kinetic-strategy/src 6 files
    - constants.rs Rust
    - kinetic.rs Rust
    - lib.rs Rust
    - shares.rs Rust
    - storage.rs Rust
    - swap.rs Rust
  - stablebond-wrapper-strategy/src 6 files

- aqua.rs Rust
- constants.rs Rust
- lib.rs Rust
- pricing.rs Rust
- shares.rs Rust
- storage.rs Rust

REMEDIATION COMMIT ID:

- a8e8d51781ea88fad79d6eba3e0e3b9b8b42b147
- c827282db1212091f30fd9a04fb05507ae3d4452
- ac7c53b1
- 284d533d3228353b25db2304d9d0a997ec97352b
- bb0e02be51d210972d2f3bb7543f7e90a55e1425
- c815116dabb87fb12df4323c445bb678875ded13
- 14faaa759913f8ea59650e8110622b3bf83b2b27
- 79bd9865f45ea5890fa686389a20228d2b5615e5
- f11e341e8e0e83b2a39554e35cc24eaa2e6f5506
- f5249025e4d35807f30ecb11b9e8b441e3dc1309
- 46d4777d30671fa03046c488cddc2cdf94804e53
- c5cfa36d8ca9885348c0160802b82948800d065a
- cdb86ec55eff61153ba4d5211f4d9102ce8d9a2d
- 5937503fccd82a44f014f91b61efefdd6a12a5f3
- 55f81bc4

Out-of-Scope: New features/implementations after the remediation commit IDs.

Assessment Summary & Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
HAL-001	Multi-Hop Reward Harvest Path Always Reverts	Medium	5.9	SOLVED 08/31/2026
HAL-002	Deposit Share Minting Dilutes Existing Shareholders	Medium	5.0	SOLVED 08/31/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-003	Harvest Yield Sniping Via Timed Deposits	Medium	5.0	SOLVED 08/31/2026
HAL-004	Idle Underlying Balance Permanently Locked	Low	3.4	SOLVED 08/31/2026
HAL-005	Self-Referential Aqua Pricing Enables Dilution	Low	3.4	RISK ACCEPTED 08/31/2026
HAL-006	Depositor Allowlist Bypass Via Wasm Hash	Low	2.5	RISK ACCEPTED 08/31/2026
HAL-007	Post-Swap Balance Check Blocks Large Withdrawals	Low	2.5	SOLVED 08/31/2026
HAL-008	Pricing Params Update Event Always Emits Zero	Low	2.5	SOLVED 08/31/2026
HAL-009	Missing New-Keeper Auth Enables Permanent Lockout	Low	2.1	SOLVED 08/31/2026
HAL-010	Immutable Kinetic Router Address Permanently Locks Funds	Low	2.0	RISK ACCEPTED 08/31/2026
HAL-011	Unbounded Slippage Bps Risks Severe Principal Loss	Low	2.0	SOLVED 08/31/2026
HAL-012	Immutable Oracle Address Risks Permanent Fund Lockup	Low	2.0	RISK ACCEPTED 08/31/2026
HAL-013	Zero-Balance VaultPos Entries Persist After Withdrawal	Informational	1.7	SOLVED 08/31/2026
HAL-014	Missing Detection For RWA Clawback Or Freeze	Informational	1.0	SOLVED 08/31/2026
HAL-015	Aqua Admin Pool Removal Redirects Strategy To Empty Pool	Informational	1.0	ACKNOWLEDGED 08/31/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-016	Unvalidated Min Out Enables Harvest Sandwich Attacks	Informational	1.0	SOLVED 08/31/2026
HAL-017	Unrevoked Router Allowance Enables Latent Drain	Informational	1.0	SOLVED 08/31/2026
HAL-018	Unbounded Decimals Value Can Brick Deployment	Informational	0.8	SOLVED 08/31/2026
HAL-019	Unvalidated Harvest Path Risks Idle Balance Misrouting	Informational	0.5	SOLVED 08/31/2026
HAL-020	Liquidity Index Tripwire Guards Wrong Contract	Informational	0.1	SOLVED 08/31/2026

Findings & Tech Details

HAL-001 SOLVED

Multi-Hop Reward Harvest Path Always Reverts

Medium · 5.9 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:M/I:N/D:N/Y:H

DESCRIPTION

`swap::soroswap::swap_exact_tokens_for_tokens` in `KineticStrategy` builds the `router_pair_for` call by passing the multi-element `path` `Vec<Address>` directly as `try_invoke_contract`'s flat argument list. The soroban-sdk conversion spreads each `path` element into its own positional argument, so a 3-element path becomes 3 positional arguments, while Soroswap's `router_pair_for` accepts exactly 2 positional `Address` arguments.

The keeper reaches this defect by submitting a `harvest` call whose `HarvestInstruction.path` contains 3 or more elements, which the documented multi-hop reward route requires for any reward token with no direct pool to the strategy's asset, for example `SOLV -> XLM -> USDC`. The argument-count mismatch traps at the host boundary and is re-panicked as `StrategyError::SoroswapPairError`, reverting the entire `harvest` call.

Code Location

`swap.rs::soroswap::swap_exact_tokens_for_tokens()`: spreads path elements as flat `try_invoke_contract` args for `router_pair_for`

RUST 78–85

```rust
let pair_address = e
            .try_invoke_contract::<Address, InvokeError>(
                router,
                &Symbol::new(e, "router_pair_for"),
                path.clone().into_val(e),
            )
            .unwrap_or_else(|_| panic_with_error!(e, StrategyError::SoroswapPairError))
            .unwrap();
```

`swap.rs`: the correctly-nested path argument in the very next call, contrasting with the defect above

RUST 99–106

```rust
let swap_args = vec![
            e,
            amount_in.into_val(e),
            amount_out_min.into_val(e),
            path.into_val(e),
            to.to_val(),

```

```
                deadline.into_val(e),
            ];
```

Impact

Every `harvest` call for a reward token that requires a 3-or-more element swap path reverts unconditionally, permanently denying the strategy the ability to harvest that reward token; the default 2-hop harvest path is unaffected.

Because the contract is non-upgradable, this denial is permanent: any reward token that requires a multi-hop route can never be harvested, resulting in a total, irrecoverable loss of all future compounding yield from that reward token.

PROOF OF CONCEPT

Code

The PoC code has been added to the existing test suite located in the `contracts/kinetic-strategy/src/test.rs` file, as a pair of test functions: `poc_multi_hop_harvest_reward_path_always_reverts` (demonstrates the failure) and `poc_multi_hop_harvest_succeeds_when_router_arity_matches_flattened_path` (differential control isolating the root cause). A supporting mock contract, `MockSwap3Arity`, was added alongside the suite's existing `MockSwap` to serve the control.

RUST

```rust
#[test]
fn poc_multi_hop_harvest_reward_path_always_reverts() {
    // `soroswap::swap_exact_tokens_for_tokens` (swap.rs:78-85) resolves the router
    // pair by calling `router_pair_for` through `try_invoke_contract` with
    // `path.clone().into_val(e)` passed as the FLAT positional argument list. soroban-sdk
    // spreads each `path` element into its own positional arg, but Soroswap's real
    // `router_pair_for(token_a: Address, token_b: Address)` (mirrored faithfully by
    // `MockSwap::router_pair_for` at test.rs:300, which takes exactly 2 `Address` params)
    // accepts exactly 2. Contrast with the very next call (swap.rs:99-106), which correctly
    // nests `path` as a single `Vec<Address>` argument.
    //
    // §A. A 2-element path (the default direct hop, [reward, asset]) spreads into exactly 2
    //     args -> matches `router_pair_for`'s arity -> succeeds. Already covered by the
    //     existing `harvest_swaps_reward_to_asset` test; not repeated here.
    // §B. A 3-element path — the documented route for a reward token with no direct pool to
    //     the strategy's asset, e.g. `SOLV -> XLM -> USDC` (lib.rs:68 comment) — spreads into
    //     3 args against a 2-arg host function. The mismatch traps at the host boundary and
    //     is re-panicked as `StrategyError::SoroswapPairError` (swap.rs:84), reverting the
    //     ENTIRE `harvest` call and rolling back the `claim_rewards` side effect that ran
    //     earlier in the same transaction.
    let f = setup(false); // reward_token != asset -> the swap leg is exercised
    let vault = Address::generate(&f.env);
    fund(&f, &vault, 100_0000000);
    f.strategy().deposit(&100_0000000, &vault);

    // Intermediate hop token standing in for XLM in the documented SOLV -> XLM -> USDC route.
    let intermediate = sac(&f.env).0;

    f.incentives().set_pending(&10_0000000);
```

```rust
    // §B — build the keeper's multi-hop harvest instruction: reward -> intermediate -> asset.
    use soroban_sdk::xdr::ToXdr;
    let multi_hop_data = crate::HarvestInstruction {
        reward_token: f.reward.clone(),
        min_out: 0,
        path: vec![&f.env, f.reward.clone(), intermediate.clone(), f.asset.clone()],
    }
    .to_xdr(&f.env);

    let res = f.strategy().try_harvest(&f.keeper, &Some(multi_hop_data));

    // The keeper's multi-hop instruction is unconditionally rejected — not because the route
    // is bad or a pool is missing, but purely because of the flattened-args defect.
    assert_eq!(
        res,
        Err(Ok(StrategyError::SoroswapPairError)),
        "multi-hop harvest must revert with SoroswapPairError due to the flattened path args"
    );

    // The whole transaction rolled back: NAV is unchanged, so the 10-token reward is neither
    // reinvested nor lost from the mock's ledger — it is simply permanently unharvestable via
    // any multi-hop route, exactly as the finding describes.
    assert_eq!(
        atoken_balance(&f, &f.strategy_id),
        100_0000000,
        "NAV must be unchanged: the multi-hop harvest never reinvests anything"
    );
}

#[test]
fn poc_multi_hop_harvest_succeeds_when_router_arity_matches_flattened_path() {
    // differential control, isolating the root cause behind the sibling PoC above
    // (`poc_multi_hop_harvest_reward_path_always_reverts`). That test shows the SYMPTOM: a
    // 3-element harvest path reverts with `SoroswapPairError` against `MockSwap`, whose
    // `router_pair_for` faithfully mirrors real Soroswap's 2-arg signature. On its own, that
    // only proves failure correlates with path length against THAT router shape — it does not
    // yet prove the CAUSE is arg-count spreading rather than, say, pair/token validation.
    //
    // This test changes exactly one thing: the router's `router_pair_for` now takes 3
    // positional `Address` args (`MockSwap3Arity`) instead of 2 — a shape no real Soroswap
    // deployment has, used here purely as an oracilloscope probe. The strategy's own code is
    // untouched, and the harvest path is byte-for-byte the same 3-element
    // `[reward, intermediate, asset]` route that reverted above. If harvest now succeeds, the
    // only variable that changed was the callee's arity — which is direct, not inferred,
    // evidence that `swap.rs:78-85` is spreading `path` into positional args whose count must
    // match whatever `router_pair_for` happens to accept, exactly as the finding claims.
    let env = Env::default();
    env.mock_all_auths();

    let (asset, _asset_admin) = sac(&env);
    let reward = sac(&env).0;
    let intermediate = sac(&env).0;

    let router_id = env.register(MockRouter, (asset.clone(),));
    let atoken = env.register(MockAToken, (router_id.clone(), asset.clone()));
    MockRouterClient::new(&env, &router_id).set_atoken(&atoken);

    let incentives_id = env.register(MockIncentives, ());

    // The only change from the sibling PoC's fixture: a router whose `router_pair_for` takes
    // 3 positional args, matching the strategy's actual (buggy) flattened call for this path.
    let swap_id = env.register(MockSwap3Arity, (asset.clone(),));
    StellarAssetClient::new(&env, &asset).mint(&swap_id, &1_000_000_0000000);
    StellarAssetClient::new(&env, &reward).mint(&incentives_id, &1_000_000_0000000);
```

```
    let keeper = Address::generate(&env);
    let init_args: soroban_sdk::Vec<soroban_sdk::Val> = (
        router_id.clone(),
        incentives_id.clone(),
        swap_id.clone(),
        keeper.clone(),
    )
        .into_val(&env);
    let strategy_id = env.register(KineticStrategy, (asset.clone(), init_args));
    let strategy = KineticStrategyClient::new(&env, &strategy_id);

    let vault = Address::generate(&env);
    StellarAssetClient::new(&env, &asset).mint(&vault, &100_0000000);
    strategy.add_depositor(&vault);
    strategy.deposit(&100_0000000, &vault);

    MockIncentivesClient::new(&env, &incentives_id).set_pending(&10_0000000);

    // Byte-for-byte the same 3-element path as the sibling PoC's failing case.
    use soroban_sdk::xdr::ToXdr;
    let multi_hop_data = crate::HarvestInstruction {
        reward_token: reward.clone(),
        min_out: 0,
        path: vec![&env, reward.clone(), intermediate.clone(), asset.clone()],
    }
    .to_xdr(&env);

    let res = strategy.try_harvest(&keeper, &Some(multi_hop_data));
    assert!(
        res.is_ok(),
        "harvest must succeed once router_pair_for's arity matches the flattened 3-element path,
got {:?}",
        res
    );

    // NAV grew by the reinvested reward (1:1 swap): a real completed swap, not a
    // silently-swallowed no-op, confirms `swap_exact_in` actually ran to completion this time.
    let atoken_bal = MockATokenClient::new(&env, &atoken).balance(&strategy_id);
    assert_eq!(atoken_bal, 110_0000000, "reward must be swapped 1:1 and re-supplied");
}
```

Execution

```
SH

cargo test --package kinetic_strategy --lib -- test::poc_multi_hop_harvest --nocapture
```

Result

```
SH

running 2 tests
test test::poc_multi_hop_harvest_reward_path_always_reverts ... ok
test test::poc_multi_hop_harvest_succeeds_when_router_arity_matches_flattened_path ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 36 filtered out; finished in 0.14s
```

Analysis

Two tests isolate the defect. The first deploys the strategy against a real-shaped 2-argument mock Soroswap router and submits a harvest using a 3-element multi-hop reward path, as the strategy's docs prescribe for tokens with no direct pool. The call reverts with `SoroswapPairError`, rolling back the whole harvest. The second test changes only the router's arity to 3 arguments; with that change, the same path succeeds and NAV grows as expected. This shows swap.rs spreads the path into positional arguments that must match the router's arity. Since `KineticStrategy` is non-upgradable, future multi-hop harvests for that reward token revert permanently.

RECOMMENDATION

To mitigate this issue, it is recommended to:

- Do not reinterpret `path` as the router call's positional argument list; build the `router_pair_for` call's arguments explicitly from only the first hop's two tokens.
- Mirror the correctly-constructed usage already present for `path` in the subsequent `swap_exact_tokens_for_tokens` call, which passes `path` as a single nested `Vec<Address>` argument rather than spreading it.

REMEDIATION COMMENT

Addressed in: a8e8d51781ea88fad79d6eba3e0e3b9b8b42b147

SOLVED: The **Defindex team** solved this finding in the specified commit by following the mentioned recommendation.

HAL-002 SOLVED

Deposit Share Minting Dilutes Existing Shareholders

Medium · 5.0 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:N/D:H/Y:N

DESCRIPTION

`deposit()` in `StableBondWrapperStrategy` mints shares proportional to the USDC `amount` transferred in, priced against `nav_before`, the NAV read before the swap, rather than against the actual NAV increase produced by the deposit's USDC-to-RWA swap. Because the Aqua swap can execute with slippage up to `slippage_bps` below the oracle-implied rate, the RWA actually received can be worth less than `amount`, yet the depositor is still credited shares as if the full `amount` had landed.

This triggers on any ordinary `deposit()` call that executes with nonzero Aqua slippage, which is normal market behavior rather than an edge case; no privileged action is required for the baseline dilution to occur on every deposit.

The loss is amplified when the keeper raises `slippage_bps` toward a looser tolerance: a single large deposit that lands near the allowed slippage limit mints the depositor shares for the full deposited amount while the strategy's actual NAV rises by only a fraction of it, transferring the shortfall from existing shareholders to the depositor.

Code Location

`StableBondWrapperStrategy.deposit()`: mints shares against amount-in instead of the swap's actual NAV delta

RUST 305-310

```
305 let min_out = pricing::min_rwa_out(&e, &config, &params, amount)?;
306         aqua::swap_exact_in(&e, &config, &config.asset, &config.rwa_asset, amount,
    min_out)?;
307
308         // Mint shares (virtual-offset, floor) on the USDC amount-in.
309         let total_shares = storage::get_total_shares(&e);
310         let minted = shares::assets_to_shares_down(amount, total_shares, nav_before)?;
```

`stablebond_wrapper_strategy_research.md`: design doc's rationale for amount-in minting, contradicted by the code

MD

```
using the USDC amount input keeps share pricing stable against short-lived swap rate noise. If
Aqua is manipulated in the split-second of the swap, the depositor gets rekt on RWA received,
```

```
but their share count reflects what they paid in. Other vault holders aren't diluted by a
favorable swap; the depositor doesn't get bonus shares from an unfavorable swap.
```

Impact

Every deposit that executes with nonzero Aqua slippage mints the depositor shares worth more than the NAV the strategy actually gained, diluting all existing shareholders' pro-rata claim on the strategy's assets. Because the degree of allowed slippage is not tightly bounded, the size of the dilution scales with how much slippage a given deposit's swap incurs, and a single large deposit under a loosely configured tolerance can transfer a substantial share of existing shareholders' principal value to the depositor in one transaction.

PROOF OF CONCEPT

Code

The PoC code has been added to the existing test suite located in the `contracts/stablebond-wrapper-strategy/src/test.rs` file, under the test function `poc_deposit_dilutes_existing_shareholders`.

RUST

```rust
// ============================================================================
//
// `deposit()` mints shares against the raw USDC `amount` transferred in, priced
// at `nav_before` (lib.rs:314-316), instead of against the USDC value the
// deposit's own Aqua swap actually delivered. When that swap executes at a rate
// worse than the oracle rate used to price `nav_before` — ordinary behavior any
// time the Aqua pool trades away from the oracle by less than `max_spread_bps`
// and `slippage_bps` allow — the depositor is credited shares for the full face
// amount while the strategy's real NAV grows by strictly less. The shortfall
// does not vanish: it is subtracted from every existing shareholder's
// redeemable balance.
//
// §A — Depositor A deposits 1,000,000 USDC while Aqua trades at parity with the
//      oracle (price 1.0). A now owns 100% of the strategy; balance() ~= 1,000,000.
// §B — The Aqua pool's price is nudged 0.8% away from the oracle (the ORACLE is
//      left untouched). 0.8% is comfortably inside both max_spread_bps (1%,
//      gates the pre-trade NAV read `deposit()` uses to price A's existing
//      position) and slippage_bps (1%, gates the swap's own min_out floor) —
//      this is a passing, everyday deposit, not an edge case or an attack on
//      either guard. It models nothing exotic: any real AMM trades a fraction
//      of a percent off the oracle mid constantly.
// §C — Depositor B deposits 500,000 USDC. B's swap actually buys RWA worth only
//      ~496,032 USDC at the (unchanged) oracle rate, but `deposit()` mints B
//      shares as though the full 500,000 had landed.
// §D — The dilution, asserted directly: A did nothing between §A and §D, yet
//      A's balance() strictly decreases. A's principal is what covers the gap
//      between what B was credited and what B actually contributed, and the
//      size of A's loss tracks B's shortfall almost exactly (not rounding
//      noise — both are on the order of thousands of USDC at this scale).
// ============================================================================
#[test]
fn poc_deposit_dilutes_existing_shareholders() {
    let f = setup(Cfg {
        max_deposit: 10_000_000 * ONE,
        max_tvl: 10_000_000 * ONE,
```

```
        max_spread_bps: 100, // 1% — the launch default (research §8)
    });

    // §A — A is first in; at parity (price 1.0) A's 1,000,000 USDC buys
    // 1,000,000 RWA 1:1, so A owns the entire strategy pre-dilution.
    let a = Address::generate(&f.env);
    fund(&f, &a, 1_000_000 * ONE);
    f.strategy().deposit(&(1_000_000 * ONE), &a);
    let a_balance_before = f.strategy().balance(&a);
    assert!(
        (a_balance_before - 1_000_000 * ONE).abs() <= 2,
        "A should hold ~1,000,000 pre-dilution, got {}",
        a_balance_before
    );

    // §B — Move ONLY the Aqua pool price (the oracle is untouched): RWA is now
    // 0.8% more expensive to buy on Aqua than the oracle says it is worth.
    let skewed_price = PRICE_SCALE * 1008 / 1000; // 1.008
    f.pool().set_price(&skewed_price);

    // §C — B deposits 500,000 USDC. Its swap buys RWA at the skewed rate, i.e.
    // ~500,000 / 1.008 ~= 496,031.7 RWA - worth ~496,031.7 USDC at the oracle
    // rate the strategy uses everywhere else to price NAV and shares. `deposit()`
    // mints B shares against the raw 500,000 `amount` regardless (lib.rs:316),
    // not against this real, lesser amount.
    let b = Address::generate(&f.env);
    let b_deposit = 500_000 * ONE;
    fund(&f, &b, b_deposit);
    let rwa_before_b = rwa_held(&f);
    f.strategy().deposit(&b_deposit, &b);
    let rwa_bought_by_b = rwa_held(&f) - rwa_before_b;
    // Oracle price is exactly 1.0, so RWA units == USDC-equivalent value directly.
    let real_value_from_b = rwa_bought_by_b;
    let shortfall = b_deposit - real_value_from_b;
    assert!(
        shortfall > 3_000 * ONE,
        "the skewed swap should leave a multi-thousand-USDC gap between what B paid \
         and what B's swap actually delivered, got shortfall {}",
        shortfall
    );

    // §D — The dilution. Minting B's shares against the pre-swap `amount` instead
    // of the post-swap NAV delta does not just shortchange B by `shortfall`: it
    // socializes that shortfall across every share outstanding, B's own newly
    // minted shares included. So B does NOT end up with a full 500,000 balance —
    // B's own price-per-share drops too — but B still walks away holding strictly
    // more than the ~496,031.7 it actually contributed, and the excess is not
    // created from nothing: it is transferred, stroop-for-stroop, out of A's
    // redeemable balance, even though A took no action at all.
    let b_balance = f.strategy().balance(&b);
    let b_over_credit = b_balance - real_value_from_b;
    assert!(
        b_over_credit > 2_000 * ONE,
        "B should walk away with a balance worth strictly more than what it actually \
         contributed ({}), got balance {} (over-credit {})",
        real_value_from_b,
        b_balance,
        b_over_credit
    );

    let a_balance_after = f.strategy().balance(&a);
    assert!(
        a_balance_after < a_balance_before,
        "A's balance must not decrease from an ordinary deposit by another party; \
         before={}, after={}",
        a_balance_before,
```

```
        a_balance_after
    );
    let a_loss = a_balance_before - a_balance_after;
    assert!(
        a_loss > 2_000 * ONE,
        "A's loss should be a multi-thousand-USDC dilution, not rounding noise, got {}",
        a_loss
    );

    // The transfer is exact, not two independent rounding artifacts: what A lost
    // is what B gained beyond its real contribution, to within 1 stroop.
    assert!(
        (a_loss - b_over_credit).abs() <= 1,
        "A's loss ({}) should equal B's over-credit ({}) to within rounding — this is a \
         transfer of value from A to B, not two unrelated pricing errors",
        a_loss,
        b_over_credit
    );
}
```

Execution

```SH
cargo test --package stablebond_wrapper_strategy --lib --
test::poc_deposit_dilutes_existing_shareholders --exact --nocapture
```

Result

```SH
running 1 test
test test::poc_deposit_dilutes_existing_shareholders ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 41 filtered out; finished in 0.15s
```

Analysis

The PoC deploys the strategy against the mock Aqua pool and oracle. Depositor A deposits while pool and oracle prices match, taking 100% ownership. The pool price is then shifted 0.8% from the oracle, a gap within both `max_spread_bps` and `slippage_bps`, so no guard blocks it. Depositor B deposits 500,000 USDC, but the skewed pool only delivers ~496,032 USDC of RWA value. `deposit()` still mints B's shares against the full 500,000 USDC paid in, not the value actually received, over-crediting B by ~2,645.50 USDC while A's balance drops by the same amount, with no action from A.

RECOMMENDATION

To mitigate this issue, it is recommended to mint shares against the strategy's post-swap NAV increase rather than the pre-swap USDC amount transferred in. Compute NAV again after the deposit's swap completes, take the delta against `nav_before`, and mint shares proportional to that realized delta, so that any execution slippage is borne by the depositor alone rather than socialized across all existing shareholders.

REMEDIATION COMMENT

Addressed in: c827282db1212091f30fd9a04fb05507ae3d4452

SOLVED: The **Defindex team** solved this finding in the specified commit by measuring the RWA actually received from the deposit's swap and minting shares against its conservatively-valued (min of oracle/Aqua-quote) realized worth relative to pre-deposit NAV, instead of against the raw USDC amount-in, so execution slippage is borne by the depositor rather than socialized to existing shareholders.

HAL-003 SOLVED

Harvest Yield Sniping Via Timed Deposits

Medium · 5.0 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:N/D:N/Y:H

DESCRIPTION

`harvest` in `KineticStrategy` re-supplies claimed rewards to Kinetic, which raises the strategy's aToken NAV without minting any new shares, while `deposit` and `withdraw` mint and burn shares strictly pro-rata to NAV at the instant they are called. No minimum holding period, vesting schedule, or exit fee exists anywhere in `deposit`, `withdraw`, or `harvest` to stop capital that arrives shortly before a harvest from capturing a share of the reward it did not help accrue.

`deposit` is gated by `is_authorized_depositor`, so the realistic actor is an allowlisted DeFindex vault, or a party controlling an instance of a keeper-allowlisted vault wasm hash, rather than an arbitrary account; `withdraw` carries no such gate. That allowlist restricts who can call `deposit`, but it adds no vesting, holding-period, or fee mechanism to `harvest` itself, so any allowlisted vault or allowlisted-hash instance can still deposit a large amount shortly before the keeper's `harvest` call, which mints it a share of `total_shares` proportional to that deposit at negligible NAV impact, then withdraw shortly after `harvest` completes, redeeming its principal plus a pro-rata share of the reward increment proportional to its share balance rather than its holding duration.

Code Location

`KineticStrategy.deposit()`: shares are minted strictly proportional to instantaneous NAV and total_shares

RUST 222–227

```rust
// Mint shares (virtual-offset, floor).
        let total_shares = storage::get_total_shares(&e);
        let minted = shares::assets_to_shares_down(delta, total_shares, nav_before)?;
        if minted <= 0 {
            return Err(StrategyError::InvalidSharesMinted);
        }
```

`KineticStrategy.harvest()`: the only no-op guard is an exact-zero check, with no minimum-economic-value threshold or vesting

RUST 278–281

```rust
let claimed = kinetic::claim_rewards(&e, &config, &instr.reward_token)?;
        if claimed == 0 {
            return Ok(()); // nothing accrued -> no-op, no revert
        }
```

`KineticStrategy.harvest()`: the re-supply raises NAV without minting any shares

RUST 298–299

```rust
// Re-supply: grows the strategy's aToken NAV without minting shares.
        kinetic::supply(&e, &config, reinvest)?;
```

Impact

An allowlisted vault or allowlisted-hash instance that times its deposit and withdrawal around a harvest captures a pro-rata share of that harvest's reward increment far exceeding its actual holding duration, at the direct expense of the strategy's long-term depositors. Long-term depositors' principal remains fully redeemable at their share balance, but their earned share of that harvest's reward is reduced by whatever the timed capital extracted.

PROOF OF CONCEPT

Code

The PoC code has been added to the existing test suite located in the `contracts/kinetic-strategy/src/test.rs` file, under the test function `poc_harvest_yield_sniping_via_timed_deposit`.

RUST

```rust
// ============================================================================
//  Harvest Yield Sniping Via Timed Deposits.
//
// `deposit` mints shares strictly pro-rata to the instantaneous NAV/total_shares ratio
// (lib.rs:222-224), and `harvest` re-supplies the claimed reward straight into NAV without
// minting any shares for that increment (lib.rs:298-299). No vesting window, minimum holding
// period, or exit fee exists anywhere in `deposit`, `withdraw`, or `harvest` to make a
// harvested increment payable only to the shares that were present while it accrued. This PoC
// shows an allowlisted depositor that deposits immediately before a keeper `harvest` and
// withdraws immediately after captures a pro-rata share of that harvest's reward proportional
// only to its capital, not to its holding duration — at the direct expense of a genuine
// long-term depositor who held the position for the entire accrual window.
// ============================================================================

#[test]
fn poc_harvest_yield_sniping_via_timed_deposit() {
    let f = setup(true); // reward_token == asset, isolates the sniping mechanic from swap math

    // Long-term LP: deposits first and is the strategy's ONLY depositor while the reward
    // accrues (off-chain inside Kinetic; the harvest below claims it all in one shot).
    let lp = Address::generate(&f.env);
    fund(&f, &lp, 100_0000000);
    f.strategy().deposit(&100_0000000, &lp);

    // Sniper: an allowlisted vault (the realistic actor per lib.rs:108-116) that deposits a
    // large amount in the same instant, immediately before the keeper's harvest — capturing
    // zero seconds of the reward's accrual period.
    let sniper = Address::generate(&f.env);
    fund(&f, &sniper, 900_0000000);
    f.strategy().deposit(&900_0000000, &sniper);
```

```
    // NAV is 1000 (100 LP + 900 sniper) the instant before harvest; shares split 100/900.
    assert_eq!(atoken_balance(&f, &f.strategy_id), 1000_0000000);

    // 10 tokens of reward accrued entirely during the LP-only period (the sniper deposited an
    // instant before harvest) and the keeper compounds it atomically into NAV.
    f.incentives().set_pending(&10_0000000);
    let data = harvest_data(&f, &f.reward, 0);
    f.strategy().harvest(&f.keeper, &Some(data));
    assert_eq!(atoken_balance(&f, &f.strategy_id), 1010_0000000);

    // Sniper exits immediately after harvest, same instant — no vesting or holding-period
    // check anywhere blocks it, and `withdraw` is not even gated by the deposit allowlist.
    let sniper_balance = f.strategy().balance(&sniper);
    f.strategy().withdraw(&sniper_balance, &sniper, &sniper);

    // Sniper contributed 900 of the 1000 pre-harvest NAV (90%) but held for ZERO duration of
    // the accrual window, yet extracts ~90% of the reward: 8.9999999 of the 10 harvested
    // tokens (exact value; 1 stroop below the theoretical 9_0000000 from the virtual-offset
    // share math's floor-rounded balance query).
    let sniper_payout = TokenClient::new(&f.env, &f.asset).balance(&sniper);
    let sniper_profit = sniper_payout - 900_0000000;
    assert_eq!(
        sniper_profit, 89_999_999,
        "sniper must walk away with ~9 of the 10 harvested tokens despite zero holding duration"
    );

    // The LP — present for the ENTIRE accrual window and the strategy's only depositor while
    // the reward accrued — is left with only ~10% of the very harvest it alone earned: its
    // balance grew from its 100 principal to only 100.9999999 (~1 of the 10 reward tokens),
    // instead of the full 110 it would have received had the sniper never participated.
    let lp_balance = f.strategy().balance(&lp);
    assert_eq!(
        lp_balance, 1_009_999_999,
        "long-term LP captures only ~1 of the 10 tokens it alone accrued"
    );
}
```

Execution

```SH
cargo test --package kinetic_strategy --lib -- test::poc_harvest_yield_sniping_via_timed_deposit
--nocapture
```

Result

```SH
running 1 test
test test::poc_harvest_yield_sniping_via_timed_deposit ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 38 filtered out; finished in 0.15s
```

Analysis

The PoC uses a mock Kinetic stack where `reward_token` equals asset, isolating the sniping mechanic from any swap. A long-term LP deposits 100 units and is the sole depositor while a reward accrues off-

chain. An allowlisted sniper deposits 900 units before the keeper's harvest, capturing none of the accrual. Harvest re-supplies the 10-unit reward into NAV without minting new shares. The sniper withdraws immediately, netting 90% of the reward for zero holding time, while the long-term LP, despite earning the entire reward alone, nets only about 1 token. Since harvest is never time-gated, this repeats on every future harvest.

RECOMMENDATION

To mitigate this issue, it is recommended to:

- Vest harvested yield linearly into NAV over a fixed window instead of crediting the reinvested reward atomically in `harvest`.
- Add a minimum holding period or a time-decayed withdrawal fee so capital arriving shortly before a harvest and exiting shortly after forfeits its disproportionate share of the increment.
- At minimum, add a keeper-side minimum-claim-value threshold beyond the existing exact-zero check, so harvests batch into economically meaningful, less frequent events that narrow the timing window.

REMEDIATION COMMENT

Addressed in: ac7c53b1

SOLVED: The **Defindex team** solved this finding across commits 83e8e65, 413f04ec, ac7c53b1 by replacing atomic harvest-profit crediting with a linear vesting bucket that is excluded from the NAV used to price deposits, withdrawals, and balances, plus hardening that vesting mechanism against window-retune and full-exit edge cases.

HAL-004 SOLVED

Idle Underlying Balance Permanently Locked

Low · 3.4 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:N/D:M/Y:N

DESCRIPTION

`vault_balance` in `KineticStrategy` computes NAV strictly from `kinetic::atoken_balance`, and no function anywhere in `lib.rs` or `storage.rs` reads or reclaims the strategy's own idle underlying-asset balance. The keeper extension surface, `set_keeper`, `add_depositor`/`remove_depositor`, and `add_allowed_hash`/`remove_allowed_hash`, contains no sweep or rescue method for a stray underlying balance either.

Any external account, including a non-adversarial one, can send SEP-41 underlying tokens directly to the strategy's contract address outside the `deposit` entry point, for example by mistake. The resulting balance sits on the contract indefinitely: it is never counted in `vault_balance`, never reflected in any share's redeemable value, and unreachable by any existing method.

Code Location

`KineticStrategy.vault_balance()`: NAV is computed solely from `atoken_balance`, with no idle-underlying read

RUST 118-126

```rust
fn vault_balance(e: &Env, config: &Config, vault: &Address) -> Result<i128, StrategyError> {
    let vault_shares = storage::get_vault_shares(e, vault);
    if vault_shares == 0 {
        return Ok(0);
    }
    let total_shares = storage::get_total_shares(e);
    let nav = kinetic::atoken_balance(e, config);
    shares::shares_to_assets_down(vault_shares, total_shares, nav)
}
```

`KineticStrategy`'s keeper extension surface: no sweep or rescue method exists anywhere in it

RUST 369-371

```rust
impl KineticStrategy {
    /// Rotates the keeper. Only the current keeper can authorize the change.
    pub fn set_keeper(e: Env, new_keeper: Address) -> Result<(), StrategyError> {
```

Impact

Underlying tokens that land on the strategy's address outside a `deposit` call via an accidental direct transfer become permanently locked: unreachable by any keeper function and unrepresented in any

depositor's share value. Because the contract has no sweep method and is not upgradable, this value cannot be recovered on-chain once stranded.

RECOMMENDATION

To mitigate this issue, it is recommended to:

- Fold the strategy's own underlying-asset balance into NAV, mirroring the idle-balance treatment already used by the StableBond strategy.
- Alternatively, add a keeper-only sweep method that re-supplies any stray idle underlying balance to Kinetic.

REMEDIATION COMMENT

Addressed in: 284d533d3228353b25db2304d9d0a997ec97352b

SOLVED: The **Defindex team** solved this finding in the specified commit by folding the strategy's idle underlying balance into NAV via a new `idle_underlying` helper used in `net_nav`, sweeping it into the Kinetic supply on harvest, and spending it first on withdrawals, matching the first alternative in the recommendation (fold idle balance into NAV) plus adopting the sweep-on-harvest mechanic from the second alternative.

HAL-005 RISK ACCEPTED

Self-Referential Aqua Pricing Enables Dilution

Low · 3.4 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:N/D:M/Y:N

DESCRIPTION

`pricing::aqua_value_usdc()` in `StableBondWrapperStrategy` derives its Aqua-side NAV cross-check from `aqua::estimate_out()`, which quotes the live spot state of the exact pool identified by `config.aqua_router`, `config.aqua_tokens`, and `config.aqua_pool_index`. `aqua::swap_exact_in()`, the function that executes the strategy's actual deposit and withdraw swaps, resolves its trading pool from the identical set of `config` fields, so the Aqua leg of `priced()`'s dual-source NAV check is not structurally separate from the pool the strategy itself trades through.

An allowlisted depositor, or an instance of a keeper-allowlisted contract hash permitted by `is_authorized_depositor`, can reach this by swapping into the pool ahead of calling `deposit()` in the same transaction, moving the pool's spot price for the RWA asset down while staying within `max_spread_bps` of the SEP-40 oracle price. Because `priced()` takes `rwa_nav = oracle_v.min(aqua_v)`, the suppressed Aqua-side quote becomes the NAV used to compute `nav_before` for the share-minting calculation that follows in the same `deposit()` call.

Code Location

`pricing.aqua_value_usdc()`: the NAV check's Aqua leg quotes the live pool

RUST 86–92

```rust
fn aqua_value_usdc(
    e: &Env,
    config: &Config,
    rwa_balance: i128,
) -> Result<i128, StrategyError> {
    aqua::estimate_out(e, config, &config.rwa_asset, &config.asset, rwa_balance)
}
```

`aqua.estimate_out()`: resolves the same `config.aqua_router` / `aqua_tokens` / `aqua_pool_index` used for the NAV quote, identical to the pool `swap_exact_in()` trades through

RUST 62–77

```rust
pub fn estimate_out(
    e: &Env,
    config: &Config,
    token_in: &Address,
    token_out: &Address,
    amount_in: i128,

```

```
) -> Result<i128, StrategyError> {
    let out = AquaRouterClient::new(e, &config.aqua_router).estimate_swap(
        &config.aqua_tokens,
        token_in,
        token_out,
        &config.aqua_pool_index,
        &amount_u128,
    );
    i128::try_from(out).map_err(|_| StrategyError::ArithmeticError)
}
```

`aqua.swap_exact_in()`: the executed `depositwithdraw` swap resolves its pool from the identical config fields

RUST 85–100

```
pub fn swap_exact_in(
    e: &Env,
    config: &Config,
    token_in: &Address,
    token_out: &Address,
    amount_in: i128,
    min_out: i128,
) -> Result<i128, StrategyError> {
    let this = e.current_contract_address();
    let amount_u128 = u128::try_from(amount_in).map_err(|_|
StrategyError::ArithmeticError)?;
    let min_out_u128 = u128::try_from(min_out).map_err(|_|
StrategyError::ArithmeticError)?;

    let in_idx = token_index(config, token_in)?;
    let out_idx = token_index(config, token_out)?;
    let pool_id = AquaRouterClient::new(e, &config.aqua_router)
        .get_pool(&config.aqua_tokens, &config.aqua_pool_index);
```

Impact

A depositor who suppresses the Aqua pool's spot price immediately before calling `deposit()` causes `nav_before` to be understated by up to `max_spread_bps`, so the shares minted for that deposit are disproportionate to the USDC actually contributed. This dilutes the strategy's other existing shareholders pro-rata, with the magnitude of each instance bounded by the configured `max_spread_bps`, up to 1000 basis points.

RECOMMENDATION

To mitigate this issue, it is recommended to:

- Stop treating a live quote from the same pool the strategy trades through as an independent price source in the dual-source NAV check.
- Add a structurally separate pricing source, such as a different venue's reserves or a time-weighted average price, that a single-transaction pool manipulation cannot move.

- Tighten the maximum allowed spread for thin RWA pools until a genuinely independent second source is in place.

REMEDIATION COMMENT

RISK ACCEPTED: The **Defindex team** made a business decision to accept the risk of this finding and not alter the contracts, stating that:

> Risk accepted. With the per-deposit cap and the oracle vs pool spread tolerance in place, the worst case from the remaining self-referential quote path is bounded to roughly 50 USD per deposit cycle, and the attacker pays swap fees to set it up. The dilution vector itself was closed separately: deposit now mints against the conservative minimum of oracle and pool value (commit c827282d) and neither deposit nor withdraw re-prices the pool after its own swap (bb0e02be, 382d66e9). We prefer keeping the two independent pricing sources as designed rather than adding a third venue for that residual bound.

HAL-006 RISK ACCEPTED

Depositor Allowlist Bypass Via Wasm Hash

Low · 2.5 AO:A/AC:L/AX:L/R:P/S:U/C:N/A:N/I:M/D:N/Y:N

DESCRIPTION

`is_authorized_depositor` in both `KineticStrategy` and `StableBondWrapperStrategy` grants deposit authorization to any caller whose deployed contract shares a keeper-allowlisted wasm code hash, with no check binding that hash to a specific deployer or factory-issued instance. Because uploaded Soroban wasm bytecode is public, any account can deploy its own instance of the identical allowlisted bytecode and satisfy this check in either contract.

The keeper reaches the vulnerable state by calling `add_allowed_hash` to allowlist the DeFindex vault's wasm hash. Any external account can then deploy its own instance of that same public bytecode, become its admin, and call `deposit` from that instance. `require_auth()` succeeds because the caller controls the instance it just deployed, and `is_authorized_depositor` matches the instance's code hash against the allowlisted hash and returns `true`.

`KineticStrategy`'s `Config` struct defines no TVL or deposit cap field at all, so nothing in that contract bounds how much such an instance can deposit. `StableBondWrapperStrategy` does define `max_tvl`, so a bypassing instance there is bounded by the same cap that governs the legitimate vault, at the cost of consuming headroom the legitimate vault needs.

Code Location

`KineticStrategy.is_authorized_depositor()` and `StableBondWrapperStrategy.is_authorized_depositor()`: identical pattern in both contracts, grants deposit authorization based solely on a wasm code-hash match, with no deployer binding

RUST

```rust
fn is_authorized_depositor(e: &Env, from: &Address) -> bool { // kinetic-strategy/src/lib.rs:108, stablebond-wrapper-strategy/src/lib.rs:112
    if storage::is_depositor(e, from) {
        return true;
    }
    match from.executable() {
        Some(Executable::Wasm(hash)) => storage::is_allowed_hash(e, &hash),
        _ => false,
    }
}
```

`KineticStrategy.deposit()`: `is_authorized_depositor` is the sole gate before a deposit proceeds

RUST 200–202

```rust
if !is_authorized_depositor(&e, &from) { // line 200
            return Err(StrategyError::NotAuthorized);
        }
```

`KineticStrategy`'s `Config` struct: confirms no `max_tvl` or deposit-cap field bounds the bypass in this contract

RUST 9–20

```rust
pub struct Config {
    /// Underlying asset managed by the strategy (e.g. USDC).
    pub asset: Address,
    /// Kinetic router — supply / withdraw / index reads.
    pub kinetic_router: Address,
    /// aToken for `asset`, resolved from the router at construction.
    pub atoken: Address,
    /// Kinetic incentives controller — reward claiming.
    pub incentives: Address,
    /// AMM router used to swap a reward token -> `asset` on harvest...
    pub swap_router: Address,
}
```

Impact

Any external account can deploy its own instance of the keeper-allowlisted wasm and deposit into either strategy as if it were the legitimate DeFindex vault, bypassing the vault's own fee and accounting layer for that position. Share accounting remains per-caller-address in both contracts, so the bypassing party only ever controls its own deposited principal and shares; it cannot touch or dilute existing depositors' funds.

In `KineticStrategy`, `Config` has no TVL cap field, so this contract itself places no ceiling on the exposure; Kinetic's own protocol-side supply caps, outside this contract's scope, may further bound the practical exposure. In `StableBondWrapperStrategy`, the bypass instead consumes `max_tvl` headroom that the legitimate vault needs, producing a temporary deposit-capacity denial-of-service against real depositors rather than an unbounded exposure.

RECOMMENDATION

To mitigate this issue, it is recommended to:

- Drop code-hash-based trust in `is_authorized_depositor` in both contracts; restrict `deposit` to the explicit per-address allowlist (`storage::is_depositor`) only.
- If hash-based allowlisting must be kept, bind the allowed hash to a specific expected deployer or factory address checked at deposit time, rather than trusting any contract instance that shares that hash.

REMEDIATION COMMENT

RISK ACCEPTED: The **Defindex team** made a business decision to accept the risk of this finding and not alter the contracts, stating that:

> Accepted as intended behavior, discussed with the assessment team during the engagement. The DeFindex vault wasm is open source and clients deploy their own vault instances, so gating depositors by vault code hash is a product requirement: every vault running our audited code should be able to plug in this strategy without us manually allowlisting each new client deployment. A contract sharing the hash is by definition running the same audited vault code, and the strategy's share accounting does not depend on which vault instance deposits.

HAL-007 SOLVED

Post-Swap Balance Check Blocks Large Withdrawals

Low · 2.5 AO:A/AC:L/AX:L/R:P/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

`withdraw()` in `StableBondWrapperStrategy` computes its returned balance by calling `vault_balance()` after the RWA-to-USDC swap has already executed, and `vault_balance()` re-runs the same spread-checked pricing path, `priced()`, that the pre-trade NAV gate uses to detect oracle-versus-Aqua manipulation. That check compares the Aqua-quoted value of the strategy's remaining RWA balance against the SEP-40 oracle price and returns `StrategyError::ExternalError` whenever the spread exceeds `max_spread_bps`, but the check cannot distinguish a manipulated price from the price impact the withdrawal's own swap just caused on the pool.

A partial withdrawal reaches this path whenever the RWA sold in the first step moves the Aqua pool's spot price for the remaining RWA balance more than `max_spread_bps` away from the oracle price. Because the `vault_balance()` error propagates through the `?` operator, the entire transaction, including the swap and the share burn, reverts under Soroban's atomicity guarantees. A full withdrawal does not trigger this path, because it drives the held RWA balance to zero and `priced()` short-circuits before reaching the spread check.

Code Location

`StableBondWrapperStrategy.withdraw()`: swaps RWA to USDC, then re-prices the remaining balance through the same spread-checked path

RUST 402–404

```rust
aqua::swap_exact_in(&e, &config, &config.rwa_asset, &config.asset, rwa_in, to_swap)?;
        ...
        let balance = vault_balance(&e, &config, &params, &from)?;
```

`pricing.priced()`: the spread check that can revert on the withdrawal's own post-swap price impact

RUST 120–125

```rust
let oracle_v = oracle_value_usdc(e, config, params, balance)?;
    let aqua_v = aqua_value_usdc(e, config, balance)?;

    if spread_bps(oracle_v, aqua_v)? > params.max_spread_bps as i128 {
        return Err(StrategyError::ExternalError);
    }
```

Impact

A partial withdrawal whose own swap moves the Aqua pool's spot price beyond `max_spread_bps` reverts entirely, so the vault cannot exit a large position in a single transaction once it exceeds the pool's available depth at the configured spread tolerance. Soroban's atomicity guarantee means no funds move and no shares burn on the reverted attempt, but the vault has no on-chain fallback and must split the withdrawal into multiple smaller transactions to succeed, which is unguided and undocumented for strategy operators.

RECOMMENDATION

To mitigate this issue, it is recommended that the trailing balance read in `withdraw()` no longer re-run the full spread-checked pricing path against the post-swap pool state. Compute the returned balance from the pre-swap NAV and the shares just burned, or fall back to an oracle-only valuation for this informational read, so the withdrawal's own legitimate price impact cannot revert a transaction that already passed the pre-trade manipulation check.

REMEDIATION COMMENT

Addressed in: bb0e02be51d210972d2f3bb7543f7e90a55e1425

SOLVED: The **Defindex team** solved this finding in the specified commit by following the mentioned recommendation.

HAL-008 SOLVED

Pricing Params Update Event Always Emits Zero

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

`set_pricing_params` in `StableBondWrapperStrategy` always calls `emit_params_update` with the literal values `0` and `0` for `old_value` and `new_value`, instead of the actual pre-update and post-update values of `max_spread_bps`, `quote_ttl_seconds`, and `slippage_bps`. The function correctly writes the new values to `params` and persists them through `storage::set_params`, but the event it emits carries no information about what changed.

The keeper reaches this defect on every call to `set_pricing_params`, since the hardcoded call is unconditional and does not depend on which of the three parameters actually changed. By contrast, `set_deposit_cap` and `set_tvl_cap` in the same file snapshot the prior value before updating and pass it alongside the new value to `emit_params_update`, confirming the pricing-parameters path is the outlier rather than a deliberate minimal-event design.

Code Location

`StableBondWrapperStrategy.set_pricing_params()`: always emits hardcoded `(0, 0)` instead of the real old and new parameter values

RUST 526-531

```rust
526 params.max_spread_bps = max_spread_bps;
527 params.quote_ttl_seconds = quote_ttl_seconds;
528 params.slippage_bps = slippage_bps;
529 storage::set_params(&e, &params);
530 emit_params_update(&e, "pricing", 0, 0);
531 Ok(())
```

`StableBondWrapperStrategy.set_deposit_cap()`: the correct pattern used elsewhere in the same file, snapshotting and emitting the real old and new values

RUST 486-489

```rust
486 let old = params.max_deposit;
487 params.max_deposit = new_cap;
488 storage::set_params(&e, &params);
489 emit_params_update(&e, "max_deposit", old, new_cap);
```

Impact

An off-chain monitoring system watching `ParamsUpdatedEvent` for field `"pricing"` always sees `{old_value: 0, new_value: 0}`, regardless of how far the keeper relaxes `max_spread_bps`, `quote_ttl_seconds`, or `slippage_bps`. This removes the only on-chain audit trail for changes to the strategy's three primary anti-manipulation controls, so a monitor cannot distinguish a benign re-submission from a deliberate weakening of these parameters.

The underlying on-chain state is not affected: `storage::set_params` persists the real values, and `pricing::nav_guarded` enforces the currently stored parameters correctly. The defect degrades observability of parameter changes, not the enforcement of the parameters themselves.

RECOMMENDATION

To mitigate this issue, it is recommended to snapshot the prior value of each of `max_spread_bps`, `quote_ttl_seconds`, and `slippage_bps` before writing the update, then emit the real old and new values, either as three separate `emit_params_update` calls or as a single event carrying all three deltas. This brings `set_pricing_params` in line with the pattern already used by `set_deposit_cap` and `set_tvl_cap` and restores on-chain auditability of every security parameter change.

REMEDIATION COMMENT

Addressed in: `c815116dabb87fb12df4323c445bb678875ded13`

SOLVED: The **Defindex team** solved this finding in the specified commit by following the mentioned recommendation.

HAL-009 SOLVED

Missing New-Keeper Auth Enables Permanent Lockout

Low · 2.1 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:C/I:N/D:N/Y:L

DESCRIPTION

`set_keeper` in both `KineticStrategy` and `StableBondWrapperStrategy` transfers the keeper role after only the current keeper calls `require_auth()`; neither implementation calls `new_keeper.require_auth()` before writing the new address to storage. The incoming keeper's ability to sign transactions, and its consent to accept the role, is never verified.

The current keeper reaches this defect on every call to `set_keeper`, whether through a typo in the `new_keeper` address, an insider threat, or a compromised keeper key. Because every keeper-gated function, `add_depositor`, `remove_depositor`, `add_allowed_hash`, `remove_allowed_hash`, `harvest`, and in `StableBondWrapperStrategy` also `pause_deposits`, `set_deposit_cap`, `set_tvl_cap`, and `set_pricing_params`, checks only the stored keeper address, an inaccessible new keeper immediately disables all of them, with no on-chain recovery in either non-upgradable contract.

Code Location

`KineticStrategy.set_keeper()`: only the old keeper authorizes; the new keeper is never required to prove access

RUST 372–384

```rust
pub fn set_keeper(e: Env, new_keeper: Address) -> Result<(), StrategyError> {
        extend_instance_ttl(&e);
        let old_keeper = storage::get_keeper(&e)?;
        old_keeper.require_auth();
        storage::set_keeper(&e, &new_keeper);
        SetKeeperEvent {
            strategy_name: String::from_str(&e, STRATEGY_NAME),
            old_keeper,
            new_keeper,
        }
        .publish(&e);
        Ok(())
    }
```

`StableBondWrapperStrategy.set_keeper()`: the same unilateral transfer pattern

RUST 447–453

```rust
pub fn set_keeper(e: Env, new_keeper: Address) -> Result<(), StrategyError> {
        extend_instance_ttl(&e);
        let mut params = storage::get_params(&e)?;
        params.keeper.require_auth();
```

```
451         let old_keeper = params.keeper.clone();
452         params.keeper = new_keeper.clone();
453         storage::set_params(&e, &params);
```

Impact

An inaccessible `new_keeper` address permanently disables every keeper-gated administrative function in the affected contract: deposit-allowlist management, `harvest`, and, in `StableBondWrapperStrategy`, deposit pausing and cap or pricing adjustment. Neither contract contains an upgrade function or an on-chain recovery path, so the freeze is total and permanent.

A lost keeper key does not block withdraw in either contract, so depositor principal is not directly locked by this defect. Because `harvest` can no longer be called once the keeper role is lost, future reward compounding also stops for the affected strategy.

RECOMMENDATION

To mitigate this issue, it is recommended to implement a two-step keeper transfer pattern in both `KineticStrategy` and `StableBondWrapperStrategy`. The current keeper should first propose the new keeper through a `propose_keeper` call, and the proposed address should then call a separate `accept_keeper` function that requires its own `require_auth()` before the role transfer takes effect. As a simpler alternative, `set_keeper` can require `new_keeper.require_auth()` within the same call, so both the outgoing and incoming keeper must co-sign the transfer.

REMEDIATION COMMENT

Addressed in: 14faaa759913f8ea59650e8110622b3bf83b2b27

SOLVED: The **Defindex team** solved this finding in the specified commit by adding `new_keeper.require_auth()` in `set_keeper` for both `KineticStrategy` and `StableBondWrapperStrategy`, requiring the incoming keeper to co-sign the role transfer.

HAL-010 RISK ACCEPTED

Immutable Kinetic Router Address Permanently Locks Funds

Low · 2.0 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:H/I:N/D:H/Y:L

DESCRIPTION

`Config` in the Kinetic strategy stores `kinetic_router`, `atoken`, and `incentives` as immutable `Address` fields set once at construction, with no setter to update them afterward. `kinetic::withdraw` (and the analogous `supply` path) calls the router directly through `KineticRouterClient` without a `try_` variant, so a revert from the router propagates as an unhandled panic that aborts the caller's entire transaction.

This is reached when the external Kinetic protocol operator migrates its router to a new contract address and the address stored in `Config` stops responding, or ships an interface-breaking change at that address. Because `Config` has no update mechanism and the strategy is non-upgradable, every subsequent call to `withdraw` or `supply` reverts against the stale router address, with no on-chain path to point the strategy at the new router.

Code Location

`Config`: `kinetic_router` / `atoken` / `incentives` are set once with no setter

RUST 11-18

```rust
pub struct Config {
    pub asset: Address,
    pub kinetic_router: Address,
    pub atoken: Address,
    pub incentives: Address,
    ...
    pub swap_router: Address,
}
```

`kinetic.rs::withdraw()`: calls the router directly, with no try_ isolation on the revert path

RUST 143-149

```rust
pub fn withdraw(e: &Env, config: &Config, amount: i128, to: &Address) -> Result<i128, StrategyError> {
    let this = e.current_contract_address();
    let amount_u128 = u128::try_from(amount).map_err(|_| StrategyError::ArithmeticError)?;
    let withdrawn =
        KineticRouterClient::new(e, &config.kinetic_router).withdraw(&this, &config.asset, &amount_u128, to);

```

```
            i128::try_from(withdrawn).map_err(|_| StrategyError::ArithmeticError)
        }
```

Impact

Once the router becomes unreachable at the address stored in `Config`, every call to `kinetic::withdraw` reverts, permanently locking existing depositors' aToken-backed value inside the strategy with no on-chain recovery, since `Config` has no setter and the contract cannot be upgraded.

Independent of the withdrawal lockup, new deposits through `kinetic::supply` also revert against the same stale router address, so the strategy stops accepting capital entirely for as long as the address remains stale.

RECOMMENDATION

To mitigate this issue, it is recommended to:

- Add a keeper-gated update mechanism for Config's Kinetic protocol addresses (`kinetic_router`, `atoken`, `incentives`) so the strategy can be pointed at a new router if Kinetic migrates.
- Replace the direct `KineticRouterClient` calls in `supply` and `withdraw` with their `try_` variants and map failures to a typed `StrategyError`, so a router-side revert degrades gracefully instead of panicking the caller's transaction.

REMEDIATION COMMENT

RISK ACCEPTED: The **Defindex team** made a business decision to accept the risk of this finding and not alter the contracts, stating that:

> Risk accepted, discussed with the assessment team. A keeper-settable router address would let a compromised keeper repoint principal flows to an attacker contract, which is a strictly worse trade than immutability. Our upgrade mechanism is redeployment: the contract is small and cheap to redeploy, vault capital can be migrated, and the current mainnet deployment is test only. If K2 ever migrates its router we ship and audit a new strategy version pointing at it.

HAL-011 SOLVED

Unbounded Slippage Bps Risks Severe Principal Loss

Low · 2.0 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:C/Y:N

DESCRIPTION

`StableBondWrapperStrategy` validates `slippage_bps`, the parameter that bounds `min_out`/`max_in` on every Aqua swap in `deposit` and `withdraw`, only against the constraint `slippage_bps < BPS_DENOMINATOR` (100%) at construction, and repeats that same loose bound in the keeper-gated `set_pricing_params`. Unlike `max_spread_bps`, which is restricted to a tight `[MIN_SPREAD_BPS, MAX_SPREAD_BPS]` range, `slippage_bps` has no dedicated maximum constant, so it can be set as high as 9,999 basis points.

The keeper reaches this by calling `set_pricing_params` with a `slippage_bps` value close to the 100% ceiling. Because `set_pricing_params` is gated only by `params.keeper.require_auth()`, exercising this path requires the keeper role rather than any arbitrary account, matching the project's own threat model, which treats the keeper as a semi-trusted role with a wider blast radius specifically for pricing-tolerance changes. That trust boundary does not remove the risk: a compromised or careless keeper key can still push `slippage_bps` to near 100% with a single call, since no on-chain maximum prevents it.

Code Location

`StableBondWrapperStrategy.__constructor()`: only checks `slippage_bps` below 100%, with no dedicated maximum

RUST 216–219

```rust
assert!(
    (slippage_bps as i128) < BPS_DENOMINATOR,
    "slippage_bps must be < 100%"
);
```

`StableBondWrapperStrategy.set_pricing_params()`: repeats the same loose bound, contrasted with the tight max_spread_bps range

RUST 519–525

```rust
if !(MIN_SPREAD_BPS..=MAX_SPREAD_BPS).contains(&max_spread_bps)
            || quote_ttl_seconds == 0
            || quote_ttl_seconds > MAX_QUOTE_TTL_SECONDS
            || (slippage_bps as i128) >= BPS_DENOMINATOR
        {
            return Err(StrategyError::InvalidArgument);
        }
```

Impact

Once `slippage_bps` is set near its unbounded ceiling, the `min_out`/`max_in` floor or ceiling computed for the next Aqua swap in `deposit` or `withdraw` no longer meaningfully constrains the execution price. A party that sandwiches that swap can capture nearly the entire value of the swapped notional, causing the strategy to realize a severe, direct loss of principal on that single transaction with no on-chain recovery, since the contract is non-upgradable.

RECOMMENDATION

To mitigate this issue, it is recommended to add a dedicated maximum constant for `slippage_bps`, analogous to `MAX_SPREAD_BPS`, that reflects a realistic worst-case execution tolerance, and enforce it both in `__constructor` and `set_pricing_params`.

REMEDIATION COMMENT

Addressed in: 79bd9865f45ea5890fa686389a20228d2b5615e5

SOLVED: The **Defindex team** solved this finding in the specified commit by following the mentioned recommendation.

HAL-012 RISK ACCEPTED

Immutable Oracle Address Risks Permanent Fund Lockup

Low · 2.0 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:C/Y:N

DESCRIPTION

`StableBondWrapperStrategy`'s `Config` struct stores the SEP-40 `oracle` address and its cached `oracle_decimals` as plain fields written once in `__constructor`, with no setter function anywhere in the contract to update them afterward. Every other mutable operating parameter, such as the keeper address or pricing tolerances, has a dedicated keeper-gated setter, but `oracle` does not.

This is reached passively, without any caller action beyond deployment: once the strategy is live, the only way `Config.oracle` could ever change is a full contract redeployment, since the contract is non-upgradable. If the Reflector oracle instance at the deployed address is deprecated, migrated to a new address, or otherwise becomes unreachable, there is no on-chain path to point the strategy at a replacement oracle.

Code Location

`StableBondWrapperStrategy` `Config`: `oracle` stored as a plain field with no setter

RUST 14-24

```rust
pub struct Config {
    pub asset: Address,
    pub rwa_asset: Address,
    pub aqua_router: Address,
    pub aqua_tokens: Vec<Address>,
    pub aqua_pool_index: BytesN<32>,
    pub oracle: Address,
    pub asset_decimals: u32,
    pub rwa_decimals: u32,
    pub oracle_decimals: u32,
}
```

`StableBondWrapperStrategy.__constructor()`: the only place `Config`, including `oracle`, is ever written

RUST 226-243

```rust
let asset_decimals = TokenClient::new(&e, &asset).decimals();
let rwa_decimals = TokenClient::new(&e, &rwa_asset).decimals();
let oracle_decimals = PriceFeedClient::new(&e, &oracle).decimals();

storage::set_config(
    &e,
    &Config {
```

```
            asset,
            rwa_asset,
            aqua_router,
            aqua_tokens,
            aqua_pool_index,
            oracle,
            asset_decimals,
            rwa_decimals,
            oracle_decimals,
        },
    );
```

Impact

If the deployed Reflector oracle instance is deprecated, migrated to a new address, or otherwise stops returning fresh prices, every function that depends on `Config.oracle` for pricing loses its price feed permanently, since no on-chain call can update that address. Because the contract cannot be upgraded, the only remediation is a full redeployment, which abandons the existing strategy's shares and TVL rather than preserving depositor positions.

RECOMMENDATION

To mitigate this issue, it is recommended to add a keeper-gated `set_oracle()` function that updates `Config.oracle` and re-reads `oracle_decimals` from the new address. Alternatively, split `Config` into an immutable core of asset and pool identifiers and a separate, keeper-mutable set of external-integration addresses, so the oracle can be replaced without requiring a contract redeployment.

REMEDIATION COMMENT

RISK ACCEPTED: The **Defindex team** made a business decision to accept the risk of this finding and not alter the contracts, stating that:

> Risk accepted, same reasoning as the router address: a keeper-settable oracle is a direct principal-level attack path (repoint pricing, drain via mispriced swaps), which is worse than the availability risk of immutability. Deposits failing closed on a dead oracle is the intended conservative behavior, withdrawals keep working. Redeployment is the upgrade mechanism and the current mainnet deployment is test only.

HAL-013 SOLVED

Zero-Balance VaultPos Entries Persist After Withdrawal

Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The Kinetic strategy's `set_vault_shares()` function in `storage.rs` unconditionally writes the `VaultPos` entry to persistent storage and extends its TTL, with no branch that removes the entry when the resulting share balance is zero. A full withdrawal leaves behind a zero-value `VaultPos` entry that keeps renewing a 120-day TTL instead of being cleared.

This path is reached through the public `withdraw()` function, which calls `set_vault_shares(&e, &from, vault_shares.checked_sub(shares_to_burn)...)`. Because `shares_to_burn` is clamped to `vault_shares` on a full withdrawal, the subtraction resolves to zero, and `set_vault_shares` stores and TTL-bumps that zero exactly as it would a nonzero balance. Reaching this call requires no special privilege beyond already being allowed to hold a position: `deposit()`'s allowlist of authorized depositor addresses or keeper-allowlisted contract-hash instances is what creates a `VaultPos` entry in the first place, and because the entry is keyed per address, one depositor's repeated deposit and withdrawal cycles always touch its own single entry rather than creating new ones. Multiplying the number of orphaned entries requires the keeper to allowlist a contract hash and an actor to deploy multiple distinct instances of it, each performing a real, non-zero deposit and withdrawal.

Code Location

`KineticStrategy.set_vault_shares()`: writes and TTL-bumps the `VaultPos` entry unconditionally

RUST 112–120

```rust
pub fn set_vault_shares(e: &Env, address: &Address, shares: i128) {
    let key = DataKey::VaultPos(address.clone());
    e.storage().persistent().set::<DataKey, i128>(&key, &shares);
    e.storage().persistent().extend_ttl(
        &key,
        PERSISTENT_LIFETIME_THRESHOLD,
        PERSISTENT_BUMP_AMOUNT,
    );
}
```

`KineticStrategy.withdraw()`: calls `set_vault_shares` with a zero balance on a full withdrawal

RUST 340–346

```rust
storage::set_vault_shares(
    &e,
    &from,
    vault_shares
```

```
345             .checked_sub(shares_to_burn)
346             .ok_or(StrategyError::UnderflowOverflow)?,
    );
```

Impact

Because the zero-value entry is never removed, it continues to consume persistent-storage rent for up to 120 days after every full withdrawal, with no benefit to the strategy or the vault. Each address allowed to hold a `VaultPos` entry, whether directly allowlisted or an instance of a keeper-allowlisted contract hash, produces one such orphaned entry per full-withdrawal cycle, and the total rent cost scales with the number of distinct allowlisted-hash instances a keeper permits.

RECOMMENDATION

To mitigate this issue, it is recommended that `set_vault_shares()` remove the `VaultPos` entry via `persistent().remove()` when the resulting share balance is zero, instead of writing and TTL-bumping a zero value. This eliminates the orphaned entries and the ledger rent they accrue after a full withdrawal.

REMEDIATION COMMENT

Addressed in: `f11e341e8e0e83b2a39554e35cc24eaa2e6f5506`

SOLVED: The **Defindex team** solved this finding in the specified commit by following the mentioned recommendation.

HAL-014 SOLVED

Missing Detection For RWA Clawback Or Freeze

Informational · 1.0 AO:S/AC:L/AX:M/R:N/S:U/C:N/A:N/I:N/D:H/Y:N

DESCRIPTION

`StableBondWrapperStrategy` prices every deposit and withdrawal from a single live read of the RWA token balance, with no stored reference value to compare it against. `pricing::rwa_balance()` returns `TokenClient::balance()` directly. `priced()` feeds that number into `nav_readonly()` and `nav_guarded()` on every call.

The RWA-issuing SAC administrator can reduce this balance directly. `clawback` removes tokens from the strategy's balance when `AUTH_CLAWBACK_ENABLED_FLAG` is set on the asset. `set_authorized` revokes the strategy's authorization when `AUTH_REVOCABLE_FLAG` is set. Neither action involves the strategy. The next deposit or withdrawal after either event reprices shares against the reduced balance, with no check for whether the reduction was accounted for.

Code Location

`pricing.rwa_balance()`: raw, unchecked token balance read

RUST 35–37

```rust
pub fn rwa_balance(e: &Env, config: &Config) -> i128 {
    TokenClient::new(e, &config.rwa_asset).balance(&e.current_contract_address())
}
```

`pricing.priced()`: consumes `rwa_balance()` at face value with no consistency check before NAV pricing

RUST 120–130

```rust
fn priced(
    e: &Env,
    config: &Config,
    params: &Params,
) -> Result<(i128, i128), StrategyError> {
    let balance = rwa_balance(e, config);
    let idle = idle_usdc(e, config);
    if balance == 0 {
        return Ok((idle, 0));
    }
    ...
```

Impact

Share pricing has no reference to what the strategy should hold. A clawback or freeze reduces `price_per_share` for every shareholder in the same step that computes it. No on-chain signal distinguishes that loss from an ordinary price move. Depositors and withdrawers who interact after the event settle at the reduced price with no alert, and nothing in the strategy's own events tells an operator that a shortfall occurred rather than a market move.
Whether the specific RWA assets in the R1-R4 launch configuration carry clawback or revocable authority cannot be verified from the strategy's own source. The severity of this gap depends on that external, per-asset fact.

RECOMMENDATION

To mitigate this issue, it is recommended to record `rwa_balance()` after each accounted deposit and withdrawal. Compare the live balance against the value implied by that record on the next interaction. Emit an alertable event when the two diverge beyond the accounted flow. Deposits and withdrawals are the only paths that legitimately change the balance. Any other decrease can only come from the asset issuer, which keeps this check free of false positives.

REMEDIATION COMMENT

Addressed in: f5249025e4d35807f30ecb11b9e8b441e3dc1309

SOLVED: The **Defindex team** solved this finding in the specified commit by recording the RWA balance implied by the strategy's own accounted flows in a new `ExpectedRwa` storage slot and comparing it against the live balance on every deposit, withdraw, and harvest, emitting a one-shot `RwaBalanceAnomalyEvent` and resyncing when the issuer's clawback or freeze causes a shortfall.

HAL-015 ACKNOWLEDGED

Aqua Admin Pool Removal Redirects Strategy To Empty Pool

• Informational · 1.0 AO:S/AC:L/AX:M/R:N/S:U/C:N/A:N/I:N/D:H/Y:N

DESCRIPTION

`StableBondWrapperStrategy` stores `aqua_router`, `aqua_tokens`, and `aqua_pool_index` as immutable `Config` fields set once at construction, with no setter to update them. Both `deposit` and `withdraw` route every swap through `aqua::swap_exact_in`, which resolves the trading pool through `AquaRouterClient::get_pool` using this same fixed tuple for the life of the non-upgradable contract.

An Aqua operations-admin or owner can call the router's `remove_pool` function to delete the pool mapping for the strategy's token pair. A standard pool's index is derived only from its fee fraction, so a subsequent, permissionless call to `init_standard_pool` for the same tokens and fee finds no existing entry and deploys a new, empty pool contract under the identical index the strategy's `Config` already holds.

Code Location

`liquidity_pool_router/storage.rs::get_pool()`: resolves the pool through a single deterministic lookup

RUST 251-258

```rust
pub fn get_pool(e: &Env, tokens: &Vec<Address>, pool_index: BytesN<32>) -> Address {
    let salt = get_tokens_salt(e, tokens);
    let pools = get_pools(e, salt);
    match pools.get(pool_index) {
        Some(data) => data.address,
        None => panic_with_error!(&e, PoolError::PoolNotFound),
    }
}
```

`liquidity_pool_router/contract.rs::remove_pool()`: deletes the pool mapping under operations-admin or owner authorization

RUST 1417-1426

```rust
fn remove_pool(e: Env, user: Address, tokens: Vec<Address>, pool_hash: BytesN<32>) {
    user.require_auth();
    require_operations_admin_or_owner(&e, &user);
    assert_tokens_sorted(&e, &tokens);

    let salt = get_tokens_salt(&e, &tokens);
    if has_pool(&e, salt.clone(), pool_hash.clone()) {
```

```
            remove_pool(&e, salt, pool_hash)
        }
    }
```

`liquidity_pool_router/pool_utils.rs::get_standard_pool_salt()`: derives the pool index from the fee fraction alone

RUST 21-28

```rust
pub fn get_standard_pool_salt(e: &Env, fee_fraction: &u32) -> BytesN<32> {
    let mut salt = Bytes::new(e);
    salt.append(&symbol_short!("standard").to_xdr(e));
    salt.append(&symbol_short!("0x00").to_xdr(e));
    salt.append(&fee_fraction.to_xdr(e));
    salt.append(&symbol_short!("0x00").to_xdr(e));
    e.crypto().sha256(&salt).to_bytes()
}
```

`liquidity_pool/contract.rs::swap()`: a freshly created pool holds zero reserves, so any swap against it panics

RUST 543-547

```rust
let reserve_sell = reserves.get(in_idx).unwrap();
let reserve_buy = reserves.get(out_idx).unwrap();
if reserve_sell == 0 || reserve_buy == 0 {
    panic_with_error!(&e, LiquidityPoolValidationError::EmptyPool);
}
```

Impact

Every subsequent `deposit` and `withdraw` call reverts with `EmptyPool` once a new, empty pool is created under the strategy's fixed pool index, because both entry points route through the same swap function against `config.aqua_router`, `config.aqua_tokens`, and `config.aqua_pool_index`. The strategy has no setter for these fields and cannot be upgraded, so depositors cannot deposit or withdraw until an external party restores liquidity to that same pool address.

Token balances the strategy already holds remain untouched and share accounting stays internally consistent throughout, so the exposure is limited to a denial of service rather than fund loss.

RECOMMENDATION

To mitigate this issue, it is recommended to:

- Add a keeper-gated function to update `aqua_router`, `aqua_tokens`, and `aqua_pool_index` so the strategy can be repointed if the pool it depends on is removed and re-created.

- Monitor pool removal and creation events for the strategy's specific token pair off-chain, so a keeper can react before depositors attempt to use a stale configuration.

REMEDIATION COMMENT

ACKNOWLEDGED: The **Defindex team** made a business decision to acknowledge this finding and not alter the contracts, stating that:

> Risk accepted. Pool removal is an Aqua admin action against a third party protocol we already monitor operationally, and Aqua withdrawals are never pausable, so the preemptive exit path holds if it ever happens. The harmful consequence (an empty pool minting cheap shares) is independently blocked by the conservative min-of-oracle-and-pool deposit valuation from commit c827282d and by the oracle vs pool spread check, which fails closed when the pool quote diverges.

HAL-016 SOLVED

Unvalidated Min Out Enables Harvest Sandwich Attacks

Informational · 1.0 AO:S/AC:M/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:H

DESCRIPTION

`harvest` in `KineticStrategy` forwards the keeper-supplied `HarvestInstruction.min_out` directly into `swap::swap_exact_in` as the reward-to-asset swap's slippage floor, with no contract-enforced lower bound. `min_out` is declared as a bare `i128` field with no minimum-value check applied to it anywhere before the swap executes.

The keeper reaches this path by submitting a `harvest` call with `min_out` set to `0`, whether through negligence or because the keeper key is compromised. `harvest` is dual-gated by `keeper.require_auth()` and an explicit `from == keeper` check, so only the keeper itself, or a party that has compromised the keeper key, can reach this path. That guard restricts who can set `min_out`, but it adds no floor on the value itself, so once a keeper-signed `harvest` call carries `min_out = 0`, the reward swap has no slippage protection at all.

Code Location

`swap.rs::swap_exact_in()`: receives `min_out` as a bare parameter with no lower-bound validation

RUST 19-31

```rust
pub fn swap_exact_in(
    e: &Env,
    config: &Config,
    path: &Vec<Address>,
    amount_in: i128,
    min_out: i128,
) -> Result<i128, StrategyError> {
    let path = path.clone();
    let deadline = e
        .ledger()
        .timestamp()
        .checked_add(1)
        .ok_or(StrategyError::UnderflowOverflow)?;
```

`lib.rs::HarvestInstruction`: `min_out` is a bare i128 field with no documented or enforced minimum

RUST 68-76

```rust
pub struct HarvestInstruction {
    /// Reward asset to claim and compound this call.
    pub reward_token: Address,
    /// Minimum acceptable output of the `reward_token -> asset` swap (slippage floor).
    pub min_out: i128,

```

```
    /// Swap route. Empty => the default direct hop `[reward_token, asset]`; a longer path
    /// handles reward assets with no direct pool to `asset` (e.g. `SOLV -> XLM -> USDC`).
    pub path: Vec<Address>,
}
```

`lib.rs::harvest()`: `instr.min_out` passes straight through to the swap with no floor check beforehand

RUST 295

```
295 swap::swap_exact_in(&e, &config, &path, claimed, instr.min_out)?
```

Impact

With `min_out = 0`, any MEV bot that observes the pending `harvest` transaction can sandwich the reward-to-asset swap: it manipulates the relevant pool price before the swap executes and resets it afterward, extracting nearly all of that harvest cycle's reward value. The loss is confined to the just-claimed reward token for that cycle; it does not reach the strategy's aToken principal position, since `harvest` never touches principal.

PROOF OF CONCEPT

Zero min_out attack (keeper compromised or front-run):

1. Keeper signs and submits `harvest(data=XDR{reward_token=KNTC, min_out=0, path=[KNTC, USDC]})`.
2. A front-running bot observes the transaction in the mempool, manipulates the KNTC/USDC Soroswap pool to make USDC extremely expensive per KNTC.
3. Harvest claim completes: strategy receives `claimed` KNTC.
4. `swap_exact_in(claimed, min_out=0)` executes: the manipulated pool returns near-zero USDC.
5. Back-runner resets the pool price, extracting the difference.
6. Strategy supplies near-zero USDC to Kinetic instead of full reward value — effectively losing all harvest proceeds.

Deadline fragility:

1. Keeper submits harvest transaction.
2. Transaction lands in a ledger that closes 2 seconds after the `timestamp()` captured in `swap_exact_in`.
3. Soroswap router checks `deadline (= T+1) >= close_timestamp (= T+2)` → fails.

4. Harvest reverts; reward tokens may accumulate in the incentives contract without being claimed.

RECOMMENDATION

To mitigate this issue, it is recommended to enforce a contract-side lower bound on `min_out` before invoking the reward-to-asset swap, either rejecting `min_out <= 0` outright or deriving a required minimum from a fixed slippage tolerance stored in `Config` as a percentage of the claimed amount. This prevents a negligent or compromised keeper from waiving slippage protection entirely on the harvest swap.

REMEDIATION COMMENT

Addressed in: 46d4777d30671fa03046c488cddc2cdf94804e53

SOLVED: The **Defindex team** solved this finding in the specified commit by adding a `harvest`-side guard that rejects `min_out <= 0` with `InvalidArgument` whenever the reward-to-asset swap branch would execute, closing off the zero-floor sandwich path while leaving the swap-free same-asset case unaffected.

HAL-017 SOLVED

Unrevoked Router Allowance Enables Latent Drain

Informational · 1.0 AO:S/AC:L/AX:M/R:N/S:U/C:N/A:N/I:N/D:H/Y:N

DESCRIPTION

`kinetic::supply` in `KineticStrategy` grants `config.kinetic_router` a SEP-41 allowance equal to the full deposited amount, with an expiration of `current_ledger_sequence + ALLOWANCE_TTL_LEDGERS`, a constant of 17,280 ledgers, approximately one day. After the router's `supply` call returns, the function never revokes any unconsumed portion of that allowance.

Every call to `deposit` reaches this code path. If the Kinetic router, whether compromised, buggy, or maliciously upgraded, pulls less than the full approved amount during its own `transfer_from` call, the unconsumed remainder stays approved for the router to pull at any later point within the roughly one-day window, entirely outside the strategy's own accounting.

Code Location

`kinetic.rs::supply()`: grants the router a full-amount allowance with a roughly one-day expiration and no post-call revoke

RUST 120–139

```rust
pub fn supply(e: &Env, config: &Config, amount: i128) -> Result<(), StrategyError> {
    let this = e.current_contract_address();
    let amount_u128 = u128::try_from(amount).map_err(|_| StrategyError::ArithmeticError)?;

    let expiration = e
        .ledger()
        .sequence()
        .checked_add(ALLOWANCE_TTL_LEDGERS)
        .ok_or(StrategyError::UnderflowOverflow)?;
    TokenClient::new(e, &config.asset).approve(&this, &config.kinetic_router, &amount, &expiration);

    KineticRouterClient::new(e, &config.kinetic_router).supply(
        &this,
        &config.asset,
        &amount_u128,
        &this,
        &0u32,
    );
    Ok(())
}
```

`kinetic.rs`: the `ALLOWANCE_TTL_LEDGERS` constant confirming the roughly one-day window

RUST 17

```
17 | const ALLOWANCE_TTL_LEDGERS: u32 = 17_280; // ~1 day
```

Impact

If the Kinetic router under-consumes the granted allowance, whether from a bug or a compromise, it can call `transfer_from` at any point within the roughly one-day window to pull up to the full unconsumed residual of the strategy's underlying asset, up to the entire amount of that `supply` call, with no corresponding action on the strategy's side. Because the contract is non-upgradable, funds pulled this way cannot be recovered on-chain.

PROOF OF CONCEPT

1. Strategy has deposited 10,000 USDC and holds aTokens. Strategy's USDC balance is approximately 0 (all supplied to Kinetic).
2. A vault calls `strategy.deposit(amount=500e7, ...)`. This triggers `kinetic::supply(500e7)`.
3. `supply()` calls `approve(strategy, kinetic_router, 500e7, seq+17280)` — grants 500 USDC allowance to router.
4. `supply()` calls `router.supply(strategy, USDC, 500e7, strategy, 0)`.
5. Due to a bug in the router, it only calls `transfer_from(router, strategy, 500e7 - 1)` — pulling 1 stroop less.
6. `supply()` returns `Ok(())` (no return value check). The deposit succeeds.
7. The router now has a residual allowance of `1 stroop` for `17,280 ledgers` (1 day).
8. At any point in the next 24 hours, the Kinetic router can call `transfer_from(router, strategy, 1)` to pull 1 stroop of USDC from the strategy — bypassing the strategy's accounting entirely.
9. If the bug were larger (e.g., router pulls `amount / 2`), the residual would be `amount / 2 = 250 USDC`, and the router could drain that amount from the strategy within the TTL window without any vault action.

More severe scenario with a compromised router:

1. Strategy calls `supply(1_000_000e7)` (1M USDC deposit).
2. `approve(strategy, router, 1_000_000e7, seq+17280)` is granted.
3. Router calls `transfer_from(router, strategy, 1)` (consuming only 1 stroop deliberately).
4. Strategy's `supply()` call is still pending... the router's `supply()` returns after this minimal transfer.
5. `kinetic::supply()` completes (router's supply returned without error).
6. Residual allowance: 999,999.9999999 USDC — live for 17,280 ledgers.
7. Within the next 24 hours, the compromised router drains the strategy's USDC balance up to 1M USDC.

RECOMMENDATION

To mitigate this issue, it is recommended to:

- Tighten the allowance expiration to the minimum needed for the router to process the supply within the same transaction, for example `sequence + 1`, instead of the current roughly one-day window.
- Explicitly revoke any residual allowance immediately after the router's `supply` call returns, regardless of how much it consumed.

REMEDIATION COMMENT

Addressed in: c5cfa36d8ca9885348c0160802b82948800d065a

SOLVED: The **Defindex team** solved this finding in the specified commit by following the mentioned recommendation.

HAL-018 SOLVED

Unbounded Decimals Value Can Brick Deployment

Informational · 0.8 AO:S/AC:L/AX:L/R:P/S:U/C:N/A:H/I:N/D:N/Y:N

DESCRIPTION

`pricing::pow10()` computes `10^n` as an `I256` via a `while i < n` loop with no upper bound on `n`. `__constructor` validates many deployment parameters, such as asset distinctness, cap positivity, and spread/TTL/slippage ranges, but never bounds `asset_decimals`, `rwa_decimals`, or `oracle_decimals`, which are read once from `TokenClient::decimals()` and `PriceFeedClient::decimals()` and cached permanently in `Config`.

The deploying party reaches this by pointing `__constructor` at a token or oracle address whose `decimals()` call returns an oversized value, whether by mistake or through a misconfigured dependency. Because `pow10` runs on every subsequent `deposit`, `withdraw`, `harvest`, and `balance` call through `pricing::priced`, `nav_guarded`, `nav_readonly`, and `expected_rwa_for_usdc`, an oversized cached decimals value causes every one of those entry points to exceed the CPU-instruction budget, and because `Config` has no setter and the contract cannot be upgraded, there is no on-chain recovery path.

Code Location

`pricing.pow10()`: computes 10^n with no upper bound on n

RUST 40–49

```rust
fn pow10(e: &Env, n: u32) -> I256 {
    let mut acc = I256::from_i128(e, 1);
    let ten = I256::from_i128(e, 10);
    let mut i = 0;
    while i < n {
        acc = acc.mul(&ten);
        i += 1;
    }
    acc
}
```

`StableBondWrapperStrategy.__constructor()`: validates many parameters but never bounds cached decimals values

RUST 204–214

```rust
assert!(asset != rwa_asset, "asset must differ from rwa_asset");
        assert!(max_deposit > 0, "max_deposit must be positive");
        assert!(max_tvl > 0, "max_tvl must be positive");
        assert!(
            (MIN_SPREAD_BPS..=MAX_SPREAD_BPS).contains(&max_spread_bps),
```

```
            "max_spread_bps out of range"
        );
        ...
        let asset_decimals = TokenClient::new(&e, &asset).decimals();
        let rwa_decimals = TokenClient::new(&e, &rwa_asset).decimals();
        let oracle_decimals = PriceFeedClient::new(&e, &oracle).decimals();
```

Impact

If the deploying party points the constructor at a token or oracle address whose `decimals()` value is large enough to exceed the CPU-instruction budget inside `pow10`'s loop, every `deposit`, `withdraw`, `harvest`, and `balance` call fails from the first post-deploy interaction onward. Because `Config` has no setter for the cached decimals values and the contract is non-upgradable, the only recovery is a fresh redeployment with corrected addresses; the defect surfaces immediately, before any user funds are deposited, rather than after TVL accumulates.

RECOMMENDATION

To mitigate this issue, it is recommended to:

- Bound `asset_decimals`, `rwa_decimals`, and `oracle_decimals` in `__constructor` to a sane maximum, for example 18, before caching them in `Config`.
- Add an explicit upper bound inside `pow10` that returns an error rather than looping without limit.

REMEDIATION COMMENT

Addressed in: cdb86ec55eff61153ba4d5211f4d9102ce8d9a2d

SOLVED: The **Defindex team** solved this finding in the specified commit by following the mentioned recommendation.

HAL-019 SOLVED

Unvalidated Harvest Path Risks Idle Balance Misrouting

Informational · 0.5 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N

DESCRIPTION

`harvest()` in `KineticStrategy` builds the swap route for converting a claimed reward into the strategy's principal asset directly from the keeper-supplied `HarvestInstruction.path`, with no check that `path.first()` equals the declared `reward_token` or that `path.last()` equals `config.asset`. The function's own comment states that the strategy's principal is never in the swap authorization and only the just-claimed reward is, but nothing in the code enforces that guarantee when a distinct `path[0]` is supplied.

The keeper, or an attacker holding a compromised keeper key, reaches this by submitting a `HarvestInstruction` whose `path[0]` is set to `config.asset` instead of the true reward token. `swap::swap_exact_in()` then derives `token_in` from `path.get(0)` and pre-authorizes a transfer of `token_in` for the claimed reward amount via `authorize_as_current_contract`, so the pre-authorized transfer becomes denominated in the strategy's principal asset rather than the reward, contradicting the invariant the code's own comment describes. Only the keeper role can author this instruction, since `harvest` is dual-gated to require both `keeper.require_auth()` and `from == keeper`.

Code Location

`KineticStrategy.harvest()`: builds the swap path from the keeper-supplied instruction with no cross-check against reward_token

RUST 284–293

```rust
let reinvest = if instr.reward_token == config.asset {
    claimed
} else {
    let path = if instr.path.is_empty() {
        vec![&e, instr.reward_token.clone(), config.asset.clone()]
    } else {
        instr.path.clone()
    };
    swap::swap_exact_in(&e, &config, &path, claimed, instr.min_out)?
};
```

`swap.swap_exact_in()`: derives `token_in` from `path[0]` and pre-authorizes a transfer with no check against the declared reward token

RUST 73–85

```rust
let token_in = path.get(0).ok_or(StrategyError::InternalSwapError)?;

        e.authorize_as_current_contract(vec![
            e,
            InvokerContractAuthEntry::Contract(SubContractInvocation {
                context: ContractContext {
                    contract: token_in,
                    fn_name: Symbol::new(e, "transfer"),
                    args: (e.current_contract_address(), pair_address,
*amount_in).into_val(e),
                },
                sub_invocations: vec![e],
            }),
        ]);
```

`kinetic.atoken_balance()`: the true NAV-counted principal lives at a separate contract address the misrouted transfer cannot reach

RUST 109–111

```rust
pub fn atoken_balance(e: &Env, config: &Config) -> i128 {
    ATokenClient::new(e, &config.atoken).balance(&e.current_contract_address())
}
```

Impact

If the strategy holds any idle balance of its principal asset when a malicious or compromised-keeper `HarvestInstruction` sets `path[0]` to that asset, the pre-authorized transfer moves that idle balance into the swap instead of the claimed reward. The true NAV-counted principal, held as a separate aToken balance at a distinct contract address, cannot be reached through this misrouted authorization, and the swap's output still returns to the strategy's own address rather than to a third party, so the realistic worst case is misrouting or destruction of an incidental idle balance rather than loss of the strategy's actual deposited funds.

RECOMMENDATION

To mitigate this issue, it is recommended to make `harvest()` validate `path.first() == Some(&instr.reward_token)` and `path.last() == Some(&config.asset)` before invoking `swap::swap_exact_in`, rejecting any `HarvestInstruction` whose path does not begin with the declared reward token.

REMEDIATION COMMENT

Addressed in: 5937503fccd82a44f014f91b61efefdd6a12a5f3

SOLVED: The **Defindex team** solved this finding in the specified commit by adding an explicit `path.first() == reward_token` / `path.last() == asset` (`len >= 2`) check inside `harvest()` before invoking `swap::swap_exact_in`, matching the recommendation; a complementary same-PR commit further hoists this check ahead of `claim_rewards`.

HAL-020 SOLVED

Liquidity Index Tripwire Guards Wrong Contract

Informational · 0.1 AO:S/AC:L/AX:M/R:F/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

`refresh_and_guard_index` in `KineticStrategy` is documented as catching the aToken's silent index fallback, but it only reads the Kinetic router's `get_current_liquidity_index`, a value from a different on-chain contract address than the aToken contract that every NAV computation actually reads through `atoken_balance`.

Every state-changing entry point, `deposit`, `harvest`, and `withdraw`, calls `refresh_and_guard_index`, which reads `kinetic::current_liquidity_index` against `config.kinetic_router`. NAV itself is computed exclusively through `kinetic::atoken_balance`, a separate cross-contract read against `config.atoken`. If the aToken's own balance computation were to regress independently of what the router's index getter reports, `refresh_and_guard_index` would still observe `live >= stored` on the router and pass, while `atoken_balance`, the value every share conversion actually depends on, could already have silently understated NAV with no tripwire catching it.

Code Location

`KineticStrategy.refresh_and_guard_index()`: the monotonicity guard reads only the router's index getter, contradicting its own doc comment's claim about the aToken

RUST 86-98

```rust
/// The index only accrues forward by protocol invariant. A live read below the stored
/// snapshot signals a regression -- most notably the aToken's silent RAY fallback when its
/// router read fails (which would make us under-report NAV). We abort rather than proceed
/// on stale math. Run on every state-changing path; `balance` skips it (read-only).
fn refresh_and_guard_index(e: &Env, config: &Config) -> Result<(), StrategyError> {
    let live = kinetic::current_liquidity_index(e, config);
    let stored = storage::get_last_index(e);
    if live < stored {
        return Err(StrategyError::ExternalError);
    }
    storage::set_last_index(e, live);
    Ok(())
}
```

`kinetic.rs`: `current_liquidity_index` and `atoken_balance` are two independent cross-contract reads against two different config addresses

RUST 104–110

```rust
pub fn current_liquidity_index(e: &Env, config: &Config) -> u128 {
    KineticRouterClient::new(e,
&config.kinetic_router).get_current_liquidity_index(&config.asset)
}

pub fn atoken_balance(e: &Env, config: &Config) -> i128 {
    ATokenClient::new(e, &config.atoken).balance(&e.current_contract_address())
}
```

Impact

If the aToken's own balance computation regresses independently of the router's reported index, the tripwire fails to abort, and `vault_balance` silently reflects understated NAV for every depositor and vault relying on it, for as long as the divergence persists. This is a monitoring gap rather than an accounting error caused by this contract's own share-conversion logic, since share formulas remain internally consistent with whatever `atoken_balance` reports.

RECOMMENDATION

To mitigate this issue, it is recommended to:

- Derive the guarded snapshot from a value provably coupled to `atoken_balance`, such as a per-unit NAV ratio or a getter the aToken itself exposes.
- If that coupling is not available, document the tripwire's narrower router-only scope explicitly and add a second, independent sanity check directly on `atoken_balance`, for example requiring NAV not to decrease call-over-call beyond a small tolerance absent an accounted withdrawal.

REMEDIATION COMMENT

Addressed in: 55f81bc4

SOLVED: The **Defindex team** solved this finding across commits fb82664f, 55f81bc4 by replacing the fallback-prone `atoken.balance()` NAV read with a pure-math `balance_of_with_index` getter fed the exact index the tripwire already guards, and extending the same live-vs-stored monotonicity check to the read-only `balance()` query.

Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.